

Solution Requirements

Date	29 September 2026
Team ID	UNM1659GenieBirds
Project Name	EduGenie – AI-Powered Learning Assistant
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should do).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	Users should be able to access EduGenie through the application's supported user-access mechanism. Authentication should protect user-specific learning information when accounts are enabled.	Medium
2	Authorization levels	Provide appropriate access levels for learners and, where applicable, administrators. Users should only access functions and information permitted for their role.	Medium
3	External interfaces	Connect the application to the AI service/API used to generate answers, explanations, summaries, quizzes, and learning recommendations. The interface should exchange requests and responses reliably.	High
4	Transactions processing	Process user prompts and feature requests, send the required input to the AI service, receive the generated result, and present the output in the application without losing the request context.	High
5	Reporting	Provide clear learning outputs such as generated answers, explanations, summaries, quiz questions/results, and recommended learning paths. Outputs should be readable on the application interface.	High
6	Business rules, etc.	Apply the application's learning workflow: accept a learning request, identify the requested feature, generate suitable educational content, and return a structured response appropriate to the learner's request.	High
7	Compliance to laws or regulations	Handle learner information responsibly, minimize unnecessary personal data collection, protect stored information, and follow applicable privacy and data-protection requirements.	High
8	Other	Support the five core EduGenie functions: Ask a Question, Explain a Topic, Generate Quiz, Summary, and Recommended Learning Path.	High

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
------	--------------	-------------------------	-------------------------------------

1	Performance & Speed	The application should respond to normal user requests promptly and avoid unnecessary processing delays.	Typical interface response should begin within a few seconds under normal load; performance should be validated through testing.
2	Scalability	The architecture should allow the application to support increased users, requests, and learning data as usage grows.	System should support horizontal/cloud scaling and increased concurrent requests without major redesign.
3	Security & Data Privacy	Protect user inputs and learning information through secure data handling, validation, controlled access, and secure communication with external services.	Use HTTPS for data in transit, protect API credentials, validate inputs, and avoid exposing sensitive information.
4	Reliability & Availability	The system should handle normal failures gracefully and provide a usable response when an AI request cannot be completed.	No critical failure during normal test scenarios; clear error handling and recovery guidance for failed requests.
5	Usability & Accessibility	The interface should be simple, readable, consistent, and easy for learners to use when asking questions and accessing generated content.	Core features should be understandable without technical knowledge and usable through the supported interface.
6	Other	Maintainability and portability should be considered through modular code, reusable components, documented dependencies, and a clear project structure.	Core modules should be independently maintainable; setup instructions and dependencies should be documented.